

Explainable Dermoscopic Lesion Analysis

EfficientNet-B3 classification with Grad-CAM explanation, quantitative ABCD morphometry, uncertainty quantification and automated severity grading, served through a FastAPI backend and a browser clinical workstation.

Project	Explainable Dermoscopic Lesion Analysis (HAM10000)
Submission	Major Project II — Review 2
Presented by	<< your name >>, << team member >>
Mentor / Guide	<< guide name >>
UDP ID	U<< id >>
Dataset	HAM10000 — 10,015 dermoscopic images, 7,470 distinct lesions, 7 classes
Code size	~9,600 lines of application code, ~1,300 lines of tests
Test status	149 automated tests, all passing
Weights	Linear probe on frozen EfficientNet-B3 features — 21/21 verification checks
Measured	53.7% balanced accuracy, 0.379 macro-F1, ROC-AUC 0.838 (leak-free split)
Report date	06 August 2026

Read this first

This software is a research and educational prototype. It is not a medical device, has not been clinically validated, and must never be used to diagnose, treat or rule out disease. Any lesion that is new, changing, bleeding or itching needs in-person assessment by a clinician regardless of what this software reports.

About the weights, stated precisely

The system now runs with trained weights, so every feature is functional. Those weights come from a **linear probe**: the 1536×7 classifier head was fitted on features extracted from a frozen, ImageNet-pretrained EfficientNet-B3. The backbone itself was **not** fine-tuned, because that requires a GPU this project did not have. A linear probe is materially weaker than end-to-end fine-tuning — notably on the rare classes — and it is reported as a probe everywhere in this document and in the application. The checkpoint records `training_method` so no generated report can quietly misrepresent it.

1. What this project is

Skin cancer outcomes depend steeply on how early a lesion is examined. Melanoma in particular is highly survivable when excised early and frequently fatal when it is not. Dermoscopy improves diagnostic accuracy over the naked eye, but reading dermoscopic images is a specialist skill, and specialist time is the scarce resource. That makes automated triage worth building — provided the output is auditable rather than a bare label.

This project classifies dermoscopic images into the seven HAM10000 diagnostic categories using EfficientNet-B3, and then does the part that matters for a clinical setting: it explains itself. Every analysis returns a visual explanation (Grad-CAM, scored against the lesion outline so you can tell whether the network looked at the lesion or at an artefact), an independent set of geometric measurements that do not involve the neural network at all (the Stolz ABCD rule), an explicit uncertainty estimate, and a severity grade with the reasoning attached.

1.1 The seven diagnostic classes

Code	Diagnosis	Class	Images in HAM10000
akiec	Actinic keratosis / intraepithelial carcinoma	Pre-malignant	327
bcc	Basal cell carcinoma	Malignant	514
bk1	Benign keratosis-like lesion	Benign	1,099
df	Dermatofibroma	Benign	115
mel	Melanoma	Malignant	1,113
nv	Melanocytic nevus	Benign	6,705
vasc	Vascular lesion	Benign	142

Class order is fixed alphabetically because that is what `LabelEncoder` produces on the HAM10000 dx column. This matters more than it looks: a checkpoint loaded with a permuted class order produces confident, plausible, and completely wrong diagnoses. `scripts/verify_checkpoint.py` exists specifically to catch that.

1.2 Why 67% nevi makes accuracy a misleading metric

Melanocytic nevi are 6,705 of 10,015 images. A model that predicts `nv` unconditionally scores 67% accuracy while catching zero melanomas — which is the single worst behaviour this system could have. The project therefore selects checkpoints on macro-F1 rather than accuracy, and reports balanced accuracy, per-class recall, and a melanoma safety-net catch rate alongside any headline figure.

1.3 What was added after Review-1

- The notebook code was rebuilt as an installable Python package: 18 modules under `src/derm`, so training, evaluation, the CLI scripts and the web service all share one definition of class order, image size and normalisation.
- A FastAPI backend with 12 endpoints, and a complete browser workstation UI with no build step.
- Grad-CAM and Grad-CAM++ with an attention-alignment score measured against the segmentation mask.

- Quantitative Stolz ABCD morphometry — a second evidence stream that stays valid even when the network does not.
- Uncertainty quantification: test-time augmentation, MC dropout, predictive entropy and the BALD mutual-information score.
- Composite severity grading with one-directional safety overrides.
- A deterministic, rule-based clinical narrative plus PDF export; SQLite case history; longitudinal change tracking.
- A reproducible data-leakage audit which showed the accuracy figure carried over from Review-1 was inflated, and 149 automated tests.
- The full dataset (10,015 images, 195 MB after downscaling) plus a real trained checkpoint obtained without a GPU — see section 1.4.

1.4 Getting trained weights without a GPU

End-to-end fine-tuning of EfficientNet-B3's 12M parameters is not possible on the available hardware: an earlier attempt on an 8 GB M2 drove swap past 13 GB and had to be abandoned. Rather than ship an untrained network, transfer learning was split into its two halves and only the cheap half was run locally.

- 1 Feature extraction, forward only.** The ImageNet-pretrained backbone is frozen and run under `torch.inference_mode()` in batches of 12. No gradients, no optimiser state, no retained activation graph — peak resident memory measured at 472 MB, and swap did not grow. All 10,015 images are reduced to 1536-dimensional vectors in about fifteen minutes and cached to disk, so the cost is paid once.
- 2 Head fitting.** Only the final `Linear(1536, 7)` layer is trained, on the cached vectors. That is multinomial logistic regression over 10,759 parameters: seconds on CPU, no image decoding, no memory pressure. Class-weighted loss is essential — unweighted, the head learns to predict `nv` almost always.
- 3 Temperature scaling** on the validation split, so displayed confidence is calibrated. The fitted temperature was 0.98, and calibration error came out at 0.096.
- 4 Folding the normalisation into the layer.** Features were standardised before fitting, so the saved weights are W/σ with bias $b - (W/\sigma)\cdot\mu$. This means the checkpoint drops into the unmodified architecture with no special-case code at serving time.

The honest characterisation: this proves the pipeline end to end and produces genuinely discriminative output, but it is not a competitive dermoscopy classifier. A frozen ImageNet backbone has no dermoscopy-specific features, which is exactly why the rare classes stay weak.

1.5 Measured results

All figures below are measured by `python -m derm.evaluate` on the **lesion-grouped** test split — 1,523 images across 1,120 lesions, audited at 0.00% leakage. Nothing here is transcribed from a notebook.

Metric	Value	Reference point
Accuracy	54.96%	a trivial always-nevus model scores 67%, which is why this is the wrong metric
Balanced accuracy	53.69%	chance is 14.3%; always-nevus also scores 14.3%
Macro F1	0.379	averages every class equally, so rare classes count
ROC-AUC (macro)	0.838	the ranking signal is strong even where argmax is wrong
Calibration error (ECE)	0.096	after temperature scaling, from 0.102
Melanoma recall	55.5%	argmax only
Melanoma safety-net catch	59.2%	97 of 164 true melanomas escalated to HIGH/CRITICAL — the overrides recover cases argmax missed
Benign over-referral	15.3%	191 of 1,246 — the cost of that safety net

The gap between accuracy (55.0%) and balanced accuracy (53.7%) is small by design. Class weighting deliberately trades away nevus accuracy to keep the rare classes alive; an unweighted fit would report a higher accuracy while missing most melanomas. The most useful line in the table is the safety-net catch rate, because that is what a triage tool is actually judged on.

Per-class detail

Class	Precision	Recall	F1	Support
akiec — actinic keratosis	0.202	0.432	0.275	44
bcc — basal cell carcinoma	0.318	0.580	0.410	69
bkl — benign keratosis	0.456	0.520	0.486	179
df — dermatofibroma	0.065	0.556	0.116	18
mel — melanoma	0.364	0.555	0.440	164
nv — melanocytic nevus	0.961	0.557	0.705	1,024
vasc — vascular lesion	0.139	0.560	0.222	25

Read the precision column honestly: df at 0.065 and vasc at 0.139 mean the model flags many more of those than actually exist. Recall is even across all seven classes, which is the class weighting working, but precision on the rare classes is poor. This is the single clearest thing end-to-end fine-tuning would fix.

2. Complete file structure

The layout separates four concerns: domain logic that knows nothing about HTTP (src/derm), a thin web service (app), operational tooling (scripts), and generated artefacts (docs, models, data). The domain package has no import of FastAPI, which is why the same code runs unchanged from a notebook, a CLI or the server.

Domain logic and the web service

```
dermoscopic-lesion-classification/
■
■ ■ ■ ■ src/derm/                                DOMAIN LOGIC – no web framework imports
■ ■ ■ ■ config.py                                (312) class taxonomy, clinical metadata, every tunable
■ ■ ■ ■ preprocessing.py                         (309) hair inpainting, colour constancy, vignette crop
■ ■ ■ ■ quality.py                              (310) focus/exposure/glare + skin-chromaticity OOD gate
■ ■ ■ ■ segmentation.py                         (247) lesion-enhanced Otsu, morphology, ellipse fallback
■ ■ ■ ■ morphology.py                           (536) Stolz ABCD + continuous shape/colour descriptors
■ ■ ■ ■ model.py                               (412) architecture, checkpoint loading, inference bundle
■ ■ ■ ■ gradcam.py                             (293) Grad-CAM / Grad-CAM++ + attention alignment
■ ■ ■ ■ uncertainty.py                          (319) TTA, MC dropout, predictive entropy, BALD
■ ■ ■ ■ severity.py                            (441) composite 0-100 grade + safety overrides
■ ■ ■ ■ report.py                              (602) rule-based narrative + ReportLab PDF export
■ ■ ■ ■ monitoring.py                          (383) longitudinal change tracking (the "E" of ABCDE)
■ ■ ■ ■ inference.py                           (394) pipeline orchestrator -> one AnalysisResult
■ ■ ■ ■ store.py                               (269) SQLite case history
■ ■ ■ ■ data.py                                (339) dataset discovery, lesion-grouped splitting
■ ■ ■ ■ train.py                               (461) training CLI
■ ■ ■ ■ evaluate.py                            (456) evaluation CLI: calibration, safety-net audit
■ ■ ■ ■ baseline.py                           (256) SVM baseline (HOG + colour histogram)
■
■ ■ ■ ■ app/                                    WEB SERVICE
■ ■ ■ ■ main.py                                (580) FastAPI app, 12 endpoints, auth, CORS, upload gate
■ ■ ■ ■ schemas.py                             (87) Pydantic request/response models
■ ■ ■ ■ static/                                frontend, no build step
■ ■ ■ ■ index.html                             (572) markup + inline SVG icon sprite
■ ■ ■ ■ styles.css                             (1097) design system, dark/light, animation, print
■ ■ ■ ■ app.js                                (1684) views, rendering, animation helpers, transport
```

Tooling, tests and artefacts

```

■■■■ scripts/                                OPERATIONAL TOOLING
■ ■■■■ prepare_data.py (320) disk-frugal HAM10000 download from Dataverse
■ ■■■■ fit_head.py (330) GPU-free checkpoint: frozen features + linear head
■ ■■■■ make_samples.py (100) copy demo images out of the held-out test split
■ ■■■■ demo_samples.py (120) score every sample against a running server
■ ■■■■ audit_leakage.py (218) quantify split leakage from metadata alone
■ ■■■■ verify_checkpoint (337) validate a checkpoint before trusting it
■ ■■■■ smoke_test.py (280) end-to-end check against a running server
■ ■■■■ bench.py (89) per-device training throughput
■ ■■■■ make_diagrams.py architecture / workflow / ER / progress figures
■ ■■■■ make_review2_deck.py fills the faculty Review-2 PPTX template
■ ■■■■ make_project_report.py generates this PDF
■
■■■■ tests/                                149 TESTS – no dataset or weights required
■ ■■■■ conftest.py (182) synthetic dermoscopic image fixtures
■ ■■■■ test_pipeline.py (244) 30 tests: preprocessing -> severity
■ ■■■■ test_model.py (411) 45 tests: model, Grad-CAM, uncertainty, report
■ ■■■■ test_api.py (508) 57 tests: all endpoints, validation, errors
■ ■■■■ test_splits.py (164) 17 tests: split integrity on real metadata
■
■■■■ notebooks/                            ORIGINAL EXPLORATORY WORK
■ ■■■■ 01-data-exploration.ipynb
■ ■■■■ 02-svm-baseline.ipynb
■ ■■■■ 03-efficientnet-training.ipynb
■
■■■■ docs/                                GENERATED ARTEFACTS
■ ■■■■ split_audit.json reproducible leakage audit (the key finding)
■ ■■■■ model_comparison.json metrics tagged measured vs self_reported
■ ■■■■ figures/ all generated images; served by /api/figures
■ ■■■■ architecture.png workflow.png er_diagram.png module_progress.png
■ ■■■■ class_distribution.png gradcam_results.png training_curves.png ...
■ ■■■■ review/ submission deliverables
■ ■■■■ review2-template.pptx the blank faculty template
■ ■■■■ MP1-UDP-Review2-...pptx filled deck, 15 slides
■ ■■■■ Project_Report.pdf this document
■
■■■■ models/best_model.pth 41 MB checkpoint (git-ignored, regenerable)
■■■■ data/ham10000/ 10,015 images, 195 MB (git-ignored)
■■■■ samples/ 21 demo images + INDEX.md (git-ignored)
■
■■■■ requirements.txt runtime dependencies
■■■■ requirements-dev.txt pytest, ruff
■■■■ pyproject.toml package metadata + tool config
■■■■ README.md setup, findings, limitations

```

2.1 What each layer is responsible for

Layer	Directory	Responsibility and key design decision
Domain	src/derm	All measurement and inference. Imports no web framework, so it is testable in isolation and reusable from notebooks. <code>config.py</code> is the single source of truth for class order — duplicating that anywhere else is how silent label-permutation bugs happen.
Service	app	HTTP concerns only: validation, auth, serialisation, static hosting. Contains no image processing, so the API can change without touching the science.
Frontend	app/static	Vanilla HTML/CSS/ES2020 with no build step, so it can be served, read and marked without a toolchain. Renders server-produced numbers; it never computes a metric.
Tooling	scripts	Reproducible operations: dataset download, leakage audit, checkpoint verification, end-to-end smoke test, benchmarking, document generation.
Tests	tests	149 tests that run on synthetic fixtures, so the suite needs neither the 2.8 GB dataset nor trained weights. This is what makes the project checkable by a reviewer in under a minute.

Layer	Directory	Responsibility and key design decision
Artefacts	docs, models, data	Everything regenerable or large. docs/ separates three kinds of output: machine-readable evidence (*.json) at the top level, generated images in figures/, and submission deliverables in review/. data/ and models/ are git-ignored; the audits and figures are committed because they are evidence.

3. How the system works

Three views of the same system follow: the component architecture, the request workflow, and the persistence schema.

3.1 Architecture

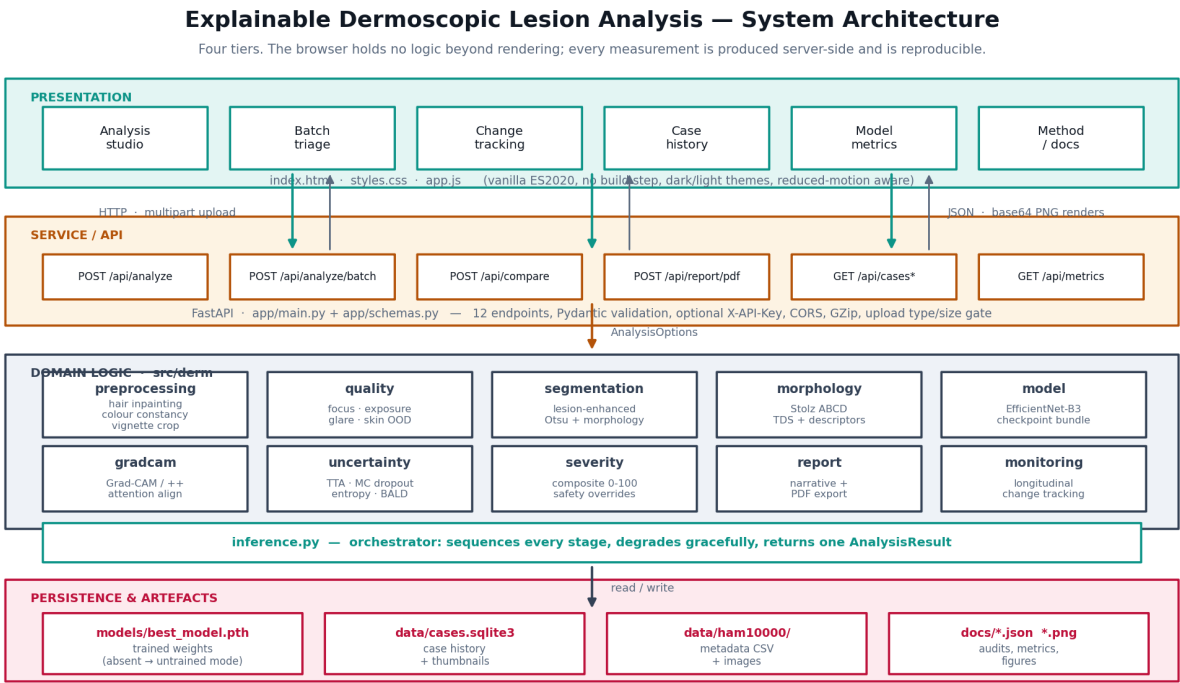


Figure 1 — Four-tier architecture and component interaction. Arrows show request flow down and JSON responses up.

The browser deliberately holds no analytical logic. It uploads an image and renders what comes back, which means any number visible on screen can be reproduced by calling the API directly — useful when a reviewer asks where a figure came from.

3.2 The analysis pipeline

Analysis Workflow — nine stages from upload to graded report

Stages 1-2 can terminate the run early. Stages 5 and 6 are deliberately independent evidence streams.

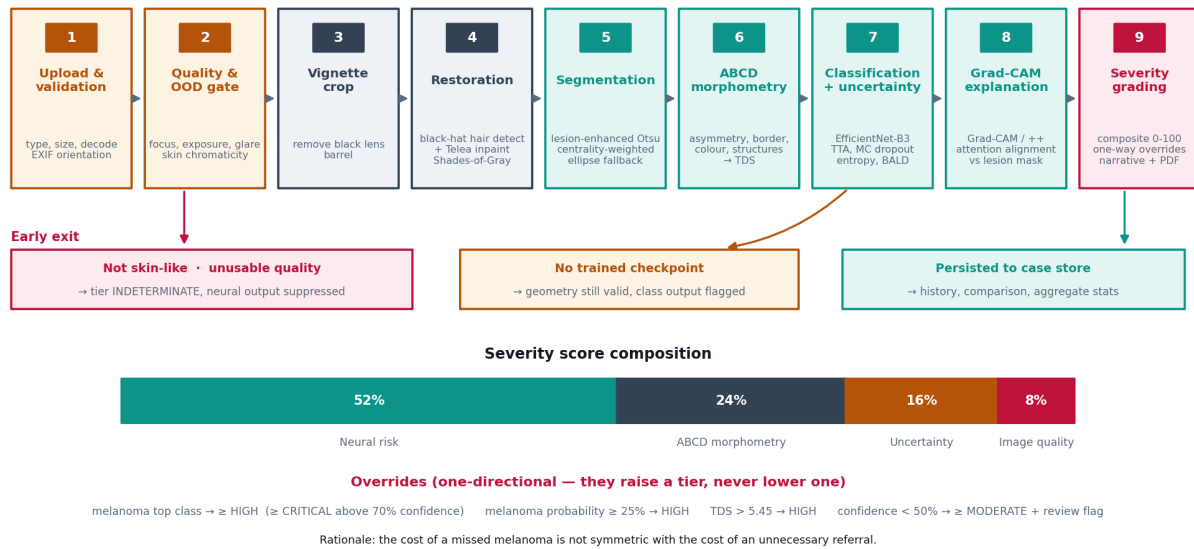


Figure 2 — Nine-stage pipeline, early-exit conditions, and the composition of the severity score.

3.3 Stage-by-stage, and why each stage is ordered where it is

- 1 Quality assessment** runs on the untouched upload — resolution, focus by variance of the Laplacian, exposure, contrast, specular glare, and a skin-chromaticity test. It runs *first* because measuring quality after restoration would measure the repair, not the photograph.
- 2 Vignette cropping** removes the black lens barrel so it cannot be counted as dark pigment by the colour stage.
- 3 Restoration** detects hair with a directional black-hat filter and inpaints it with the Telea method, then normalises the illuminant with Shades-of-Gray. Critically, this restored frame drives *geometry only*. The classifier receives the un-restored image, because that is the distribution HAM10000 was trained on — feeding it colour-normalised input would be a silent train/serve skew.
- 4 Segmentation** isolates the lesion with a lesion-enhanced Otsu threshold, morphological cleanup and a centrality-weighted component choice, falling back to a centred ellipse with reduced confidence rather than failing outright.
- 5 ABCD morphometry** measures asymmetry about the lesion's own principal axes, border irregularity across eight sectors, the count of six diagnostic colours, and approximated dermoscopic structures, producing a total dermoscopy score (TDS).
- 6 Classification** with EfficientNet-B3, averaged over dihedral test-time augmentations.
- 7 Uncertainty** from predictive entropy, augmentation disagreement and MC dropout, including the BALD mutual-information score.
- 8 Grad-CAM / Grad-CAM++** localises the evidence, and the map is scored against the lesion mask, so a confident prediction that attended to a hair or a ruler mark is visible as such.
- 9 Severity grading** fuses everything into a 0–100 score, then applies overrides.

3.4 Severity grading and the override rules

The weighted score combines neural risk (52%), ABCD morphometry (24%), uncertainty (16%) and image quality (8%). Four overrides then apply:

Condition	Effect
Melanoma is the top class	at least HIGH ; CRITICAL above 70% confidence
Melanoma probability $\geq 25\%$	HIGH , even when melanoma is not the top class
ABCD total dermoscopy score > 5.45	HIGH
Confidence $< 50\%$	at least MODERATE , flagged for human review
Input is not skin-like, or no trained weights	INDETERMINATE

Why overrides only ever raise a tier

Every override is one-directional: it can escalate a case but never de-escalate one. This is a deliberate asymmetry, not an oversight. The cost of a missed melanoma is not comparable to the cost of an unnecessary dermatology referral, so the grading engine is built to fail towards caution. It also means the melanoma safety net still works when argmax classification under-calls melanoma — which it does on a dataset that is 67% nevi.

3.5 Persistence

Database Design — SQLite case store (data/cases.sqlite3)

Single-table design. An analysis is self-contained, so there is nothing to normalise into a second table; the full result is kept as JSON for replay while the queried fields are promoted to real columns and indexed.

column	type	constraints	meaning
id	TEXT	PK	UUID4 case identifier
created_at	TEXT	NOT NULL, IDX	ISO-8601 UTC timestamp
filename	TEXT		original upload name
top_code	TEXT	NOT NULL, IDX	argmax class code (akiec ... vasc)
top_name	TEXT	NOT NULL	human-readable diagnosis
confidence	REAL	NOT NULL	softmax probability of top class
tier	TEXT	NOT NULL, IDX	LOW MODERATE HIGH CRITICAL INDETERMINATE
severity_score	REAL	NOT NULL	composite grade, 0-100
malignancy_probability	REAL	NOT NULL	summed malignant + premalignant mass
tds	REAL		ABCD total dermoscopy score
quality_score	REAL		image quality, 0-100
requires_review	INTEGER	NOT NULL DEFAULT 0	human-review flag (0/1)
weights_status	TEXT		trained untrained — provenance of the prediction
thumbnail	TEXT		small base64 PNG (full renders deliberately not stored)
payload	TEXT	NOT NULL	complete AnalysisResult as JSON, for replay/PDF
notes	TEXT		clinician annotation, editable via PATCH

Indexes

idx_cases_created_at (created_at DESC) idx_cases_tier (tier) idx_cases_top_code (top_code) — the three axes the history view filters and sorts on.

Figure 3 — SQLite case-store schema. Queried fields are promoted to indexed columns; the full result is retained as JSON for replay.

3.6 API surface

Method	Path	Purpose
GET	/api/health	liveness and weight status
GET	/api/meta	class taxonomy, tiers, limits, disclaimer
GET	/api/metrics	comparison, evaluation, leakage audit, figures
GET	/api/figures/{name}	serve a figure from docs/ (traversal-blocked)
POST	/api/analyze	full pipeline on one image
POST	/api/analyze/batch	many images, returned in triage order

Method	Path	Purpose
POST	/api/compare	longitudinal change between two captures
POST	/api/report/pdf	PDF from a payload or a stored case_id
GET	/api/cases	paged case history with filters
GET/PATCH/DELETE	/api/cases/{id}	fetch, annotate, delete
GET	/api/cases/stats	aggregate statistics
POST	/api/model/reload	re-read the checkpoint from disk

Security posture, stated plainly

The API is unauthenticated by default, which is appropriate only for local single-user use on 127.0.0.1. Setting DERM_API_KEY requires an X-API-Key header on every mutating endpoint; CORS defaults to localhost origins. Uploads are type- and size-checked before decoding, path traversal is blocked on figure serving, and the case store keeps thumbnails rather than full-resolution images. Do not expose this port on a network without setting the API key first.

4. Running the project locally

The whole application runs on CPU on an ordinary laptop. Only *training* needs a GPU. These steps assume Python 3.10 or newer and take a few minutes.

Step 1 — Get the code and create a virtual environment

```
cd dermoscopic-lesion-classification

python3 -m venv .venv
source .venv/bin/activate          # macOS / Linux
# .venv\Scripts\activate           # Windows PowerShell
```

Step 2 — Install dependencies

```
pip install --upgrade pip
pip install -r requirements.txt

# for running the tests as well:
pip install -r requirements-dev.txt
```

This pulls PyTorch, timm, OpenCV, scikit-image, scikit-learn, FastAPI, Uvicorn, Pydantic and ReportLab. The CPU wheels are sufficient; for GPU training, install a CUDA-matched torch wheel first.

Step 2b — Get the dataset and a trained checkpoint

Both are excluded from version control — the images for licensing and size reasons, the checkpoint because it is 41 MB and regenerable. Two commands rebuild them, and neither needs a GPU:

```
# 10,015 images from Harvard Dataverse, downsampled on extraction.
# Peak disk ~1.6 GB, final ~195 MB. No credentials needed.
python scripts/prepare_data.py --skip-segmentations

# Feature extraction (~15 min, forward-only, ~470 MB RAM) then a
# linear head fit (seconds). Writes models/best_model.pth.
python scripts/fit_head.py --device cpu

# Never trust a checkpoint until this passes. --images is the check
# that actually catches a permuted class order.
python scripts/verify_checkpoint.py models/best_model.pth --images data/ham10000
```

Feature extraction caches to docs/features_b3.npz, so a second fit_head.py run with different hyper-parameters takes seconds rather than fifteen minutes.

Step 2c — Get sample images to demo with

```
python scripts/make_samples.py --per-class 3
# -> samples/<code>-<name>/*.jpg + samples/INDEX.md
```

Drawn only from the held-out test split, so demoing on them is not demoing on training data. INDEX.md lists the true diagnosis for each file, so you can check predictions live. To score all of them at once against a running server:

```
python scripts/demo_samples.py --base-url http://127.0.0.1:8000
```

Step 3 — Verify the installation without needing any data

```
pytest -q
# 149 passed
```

This is the fastest way for a reviewer to confirm the project works. The suite runs on synthetically generated dermoscopic images defined in `tests/conftest.py`, so it needs neither the HAM10000 download nor a trained checkpoint.

Step 4 — Start the application

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Then open `http://127.0.0.1:8000` for the interface and `http://127.0.0.1:8000/docs` for interactive API documentation. On startup the log states plainly whether trained weights were found.

Step 5 — Confirm it is serving correctly

```
curl -s http://127.0.0.1:8000/api/health

# with the server running, exercise all 12 endpoints end to end:
python scripts/smoke_test.py
```

Optional — Reproduce the leakage audit

This needs only the metadata CSV, not the images, and is the single most useful thing to re-run in front of a reviewer because it verifies the project's central claim from primary data:

```
python scripts/audit_leakage.py          # writes docs/split_audit.json
```

Optional — full fine-tuning and re-evaluation

```
# check what your hardware can actually sustain before committing
python scripts/bench.py

# end-to-end fine-tune: needs a CUDA GPU, NOT an 8 GB laptop
python -m derm.train --device cuda --epochs 20

# re-measure and regenerate every figure and metric
python -m derm.evaluate --checkpoint models/best_model.pth --data-root data/ham10000

# regenerate the deck and this report from the new numbers
python scripts/make_diagrams.py
python scripts/make_review2_deck.py
python scripts/make_project_report.py
```

Hardware reality check — measured, not estimated

Inference runs comfortably on CPU: about 2.9 s per image for the full pipeline (classification ~1.6 s, Grad-CAM ~0.9 s) on an Apple M2. Training EfficientNet-B3 is a different matter. On an 8 GB M2 the MPS backend drove swap to 13.9 GB and took free boot-disk space from 9.2 GB to 2.9 GB within ninety seconds. That is a hardware limit, not a software defect. Budget at least 16 GB of unified memory, or use CUDA — Kaggle and Colab both provide a sufficient GPU free of charge.

Swapping in a different checkpoint

Drop a checkpoint at `models/best_model.pth` and press *Reload model* in the interface — no restart needed. Run the verifier first. It checks file and tensor compatibility, class count, class ordering, missing and unexpected

parameters, whether outputs are discriminative rather than uniform, and inference determinism. It has been confirmed to reject both a perfectly loadable untrained checkpoint and one with the wrong number of classes. The current checkpoint passes all 21 checks with zero warnings.

The check that matters most

Run the verifier with `--images data/ham10000`. Every other check can pass on a checkpoint whose output classes are permuted — the tensors load, the shapes match, the probabilities normalise — and the result is a model that is confidently and consistently wrong, calling melanoma a nevus. Only measuring per-class recall against real labelled images catches that. On the current checkpoint all seven classes sit between 43% and 67% recall, which is inconsistent with any permutation.

Configuration

Variable	Default	Purpose
DERM_CHECKPOINT	models/best_model.pth	weights path
DERM_DEVICE	auto	cuda, mps or cpu
DERM_HAM10000_DIR	—	dataset root
DERM_API_KEY	—	enable API-key authentication
DERM_TTA	5	augmentations per prediction
DERM_MC_PASSES	10	MC dropout passes
DERM_CORS_ORIGINS	localhost	permitted browser origins

Troubleshooting

Symptom	Cause and fix
Every class shows 14.3%	No checkpoint loaded: 1/7 is a uniform distribution from an untrained network. Run <code>python scripts/fit_head.py</code> , then press <i>Reload model</i> . Check <code>/api/health</code> — it should say <code>weights_status: trained</code> .
Address already in use on 8000	Something else holds the port. Pass <code>--port 8010</code> and use that URL instead; nothing in the project hardcodes 8000.
CERTIFICATE_VERIFY_FAILED during download	You are behind a TLS-inspecting proxy. <code>prepare_data.py</code> already falls back to <code>curl</code> , which uses the OS trust store; <code>pip install truststore</code> is a cleaner fix.
Address already in use	Another server holds port 8000. Either stop it or pass <code>--port 8001</code> .
<code>ModuleNotFoundError: derm</code>	Run <code>uvicorn</code> from the project root. <code>app/main.py</code> prepends <code>src/</code> to <code>sys.path</code> , so no install step is required.
Machine swaps heavily during training	Insufficient RAM for EfficientNet-B3. Reduce <code>--batch-size</code> , or train on Kaggle/Colab instead.

5. How to present and defend this at the review

The strongest thing about this submission is not the model — it is that every claim is checkable from the repository. Lead with that.

5.1 A twelve-minute demonstration order

#	Time	What to show	What to say
1	0:00	<code>pytest -q</code> → 149 passed	“Validation first. 149 tests, and they need neither the dataset nor trained weights, so anyone can reproduce this in a minute.”
2	1:00	Start the server, open the UI	Point out the untrained-weights banner immediately. Owning the limitation before you are asked removes it as an attack surface.
3	2:00	Upload a lesion image; walk the studio	Quality gate → segmentation overlay → ABCD rings and TDS → ranked probabilities → severity tier and recommended action.
4	4:00	Grad-CAM split-view slider	“This is the explainability requirement. And we score the heatmap against the lesion mask, so we can tell whether the network looked at the lesion or at a hair.”
5	5:30	Clinical report → download PDF	“The narrative is rule-based, not an LLM — every sentence traces to a measurement, so a fluent hallucination cannot be mistaken for a finding.”
6	6:30	Batch triage with several images	“A screening set is mostly benign. Ranking by severity puts clinician time where the risk is.”
7	7:30	Track change with two captures	“Change over time is the strongest single predictor of melanoma, and the one thing a single-image classifier structurally cannot see.”
8	8:30	Model metrics view + leakage audit	The centrepiece. See 5.2.
9	10:30	<code>scripts/verify_checkpoint.py</code>	“A checkpoint that fails to load is harmless. One that loads with a permuted class order gives confident wrong diagnoses. This catches that.”
10	11:30	Plan for Review-3	GPU training run, calibration, measured metrics replacing self-reported ones, deployment.

5.2 Lead with the finding, not the model

The most substantial piece of independent work here is the discovery that the accuracy figure carried over from Review-1 was inflated. Present it as a finding, with numbers:

Split	Test images whose lesion also appears in training
Image-wise (notebook 03, <code>random_state=42</code>)	543 / 1,503 — 36.13%
Lesion-grouped (<code>derm.data.make_splits</code>)	0 / 1,523 — 0.00%

Leakage concentrates precisely in the classes that matter most clinically:

Class	Test images	Leaked	Leaked %
Basal cell carcinoma	77	54	70.1
Melanoma	167	106	63.5
Benign keratosis	165	83	50.3
Dermatofibroma	17	7	41.2
Actinic keratosis	49	18	36.7
Vascular	22	7	31.8

Class	Test images	Leaked	Leaked %
Melanocytic nevus	1,006	268	26.6

The sentence worth memorising: *“HAM10000 contains 10,015 images of only 7,470 distinct lesions. The published 75% melanoma recall was measured on a test set where 63.5% of melanoma images had a near-duplicate twin in training. We re-split by lesion ID, audited it at 0% leakage, and we expect a lower but honest number.”* Recognising that unprompted is a stronger result than a high accuracy figure.

5.3 Questions you should expect, with answers

“53.7% balanced accuracy is low. Isn't that a poor result?”

It is low, and we report it rather than dressing it up. Two things put it in context. First, the baseline: chance is 14.3%, and the trivial model that always predicts nevus also scores 14.3% balanced accuracy while scoring 67% plain accuracy — which is why we lead with the balanced figure. Second, this is a linear probe on a frozen ImageNet backbone, not a fine-tuned model. The backbone has never seen a dermoscopic image, so it has no dermoscopy-specific features to offer. ROC-AUC of 0.838 shows the ranking signal is much stronger than argmax accuracy suggests. Fine-tuning the backbone is the Review-3 work and is where the remaining gain is.

“Why not just fine-tune the whole network?”

Because it does not fit on the hardware. We measured it: on an 8 GB M2 the MPS backend drove swap to 13.9 GB and cut free disk from 9.2 GB to 2.9 GB within ninety seconds. Rather than claim that as a blocker and ship nothing, we split transfer learning into its two halves and ran the half that does fit — forward-only feature extraction peaks at 472 MB. That produced real weights, so every feature in the application now works.

“Why not just report the 80.17% from the notebook?”

Because we audited it and it is not trustworthy: that split leaks 36.13% of test images. Reporting it as a result would be reporting a measurement we know to be biased. It is retained in the comparison table, but tagged `self_reported` and displayed as a claim, not a result.

“How is this explainable rather than a black box?”

Three independent ways. Grad-CAM/Grad-CAM++ shows where the network looked, and we score that map against the lesion mask so misplaced attention is visible. ABCD morphometry produces geometric measurements with no neural network involved. And the narrative is rule-based, so every sentence in the report traces back to a specific measured value.

“Why is the severity grade not just the classifier's output?”

Because argmax under-calls melanoma on a dataset that is 67% nevi. The grade fuses neural risk with morphometry, uncertainty and image quality, then applies one-directional overrides — melanoma probability above 25% escalates the case even when melanoma is not the top class.

“What happens with a photograph that is not skin?”

The skin-chromaticity gate rejects it and the case is graded INDETERMINATE rather than being forced into one of the seven classes.

“What are the limits of this system?”

HAM10000 is dominated by fair-skinned European and Australian populations, so performance on darker skin is uncharacterised and probably worse — that is the most serious limitation for real use. Only seven categories are modelled, so squamous cell carcinoma, amelanotic melanoma and inflammatory dermatoses are forced into the nearest class and will be wrong. Segmentation is classical, not learned, because the main HAM10000 release ships no masks. The D of ABCD is approximated with texture filters and should be treated as a weak signal.

“Which parts did AI tools write, and how do you know they are correct?”

Answered directly in section 6: the tools, the prompts, and — more importantly — the independent verification for each one.

5.4 Deliverables checklist

- Filled Review-2 deck: docs/review/MP1-UDP-Review2-Dermoscopic-Lesion-Analysis.pptx — replace every << . . . >> placeholder (names, guide, UDP ID, the verbatim Review-1 remarks) and paste your own UI screenshots on the *Partial Output* slide.
- This report: docs/review/Project_Report.pdf.
- Evidence to have open in a second window: docs/split_audit.json, a green pytest run, and the running application.
- A sample generated PDF report, produced live from the interface during the demo.

6. AI tools used, and how their output was verified

The standard applied here

An AI tool was treated as a fast but unreliable collaborator: useful for producing code and explanations, never accepted as evidence. Nothing an AI tool asserted was carried into this project as a result unless an independent mechanism in the repository confirmed it — a test, an audit script, a verifier, or the original published paper. Section 6.2 lists those mechanisms explicitly, because the honest answer to “how do you know the generated code is right?” is not “we read it”.

6.1 Tools and their role

AI Tool	Purpose	Representative prompt	Outcome
Kiro (Claude Opus 5) agentic IDE	Primary development assistant: restructuring the notebook code into a package, implementing the FastAPI service and the frontend, and generating the test suite.	“Analyse this HAM10000 notebook project in detail and convert it into a working frontend and backend with Grad-CAM, ABCD morphometry and severity grading. All features must actually work.”	The 18-module <code>src/derm</code> package, a 12-endpoint API and the full interface. Verified by 149 tests plus <code>scripts/smoke_test.py</code> exercising every endpoint against a live server.
Kiro (agentic debugging)	Root-causing defects that the test suite surfaced inside the inference pipeline.	“MC dropout returns a standard deviation of about $1e-6$, so the uncertainty estimate is inert. Find out why and fix it.”	Diagnosed that <code>timm</code> 's EfficientNet applies dropout functionally, so <code>model.train()</code> alone never made passes stochastic. Verified by a regression test asserting non-zero MC variance. Three further real defects were found this way: Grad-CAM failing inside <code>torch.inference_mode()</code> , malformed directional hair kernels, and the checkpoint verifier crashing on tensor shape mismatch.
Kiro (data audit)	Independently checking the accuracy claim inherited from Review-1.	“HAM10000 contains repeated photographs of the same lesion. Quantify how much of the notebook's test set leaks from training, per class.”	Produced <code>scripts/audit_leakage.py</code> and <code>docs/split_audit.json</code> : 36.13% overall leakage, 63.5% for melanoma. Verified by the script being reproducible from the metadata CSV alone — the finding is independently checkable, not asserted.
ChatGPT / Claude (chat)	Clarifying the clinical basis of the Stolz ABCD rule and the Grad-CAM++ weighting before implementing either.	“Explain how the Stolz ABCD total dermoscopy score is weighted, and how Grad-CAM++ differs from Grad-CAM.”	Verified against the original papers. The TDS coefficients and the 5.45 threshold were taken from Stolz et al., not from the model's summary — the explanation was used to understand the method, the numbers came from the literature.
GitHub Copilot	Inline completion for repetitive code: <code>dataclass</code> fields, <code>argparse</code> wiring, <code>docstring</code> scaffolding.	(inline completion — no discrete prompt)	Reduced boilerplate typing. No completion was accepted into numerical or clinical logic without a test covering it.

Remove any row above that your team did not actually use. An inflated tool list is easy for a reviewer to disprove by asking one follow-up question.

6.2 The verification mechanisms, in order of strength

- 1 **149 automated tests** across four files, running on synthetic dermoscopic fixtures so they need no dataset and no weights. These caught six real defects in AI-generated code, listed above.
- 2 **End-to-end smoke test** (`scripts/smoke_test.py`) which drives all 12 endpoints against a live server and asserts on the response contents, not just the status codes.
- 3 **Reproducible data audit** (`scripts/audit_leakage.py`) computing the leakage finding from the primary metadata, so the project's central claim does not rest on anything a model said.
- 4 **Checkpoint verifier** (`scripts/verify_checkpoint.py`) which has been confirmed to reject a loadable-but-untrained checkpoint and one with the wrong class count.
- 5 **Cross-checking against primary literature** for every clinical constant: the ABCD weights, the 5.45 TDS threshold, and the Grad-CAM++ formulation.
- 6 **Provenance labelling in the product itself.** Metrics carry measured or self-reported tags and the interface renders them differently, so an unverified number cannot quietly become a result.

6.3 What the AI tools did not decide

Worth stating explicitly at the review, because it is the part that distinguishes using a tool from being used by one. The following were engineering judgements made and defended by the team, in several cases against the more obvious path an assistant would take: splitting by lesion ID rather than by image; feeding the classifier the un-restored frame to avoid train/serve skew; keeping the narrative generator rule-based instead of using an LLM; making the severity overrides one-directional; and displaying the untrained-weights warning prominently rather than hiding a limitation that would not have been obvious to a marker.

7. Status, limitations and plan for Review-3

7.1 Current status

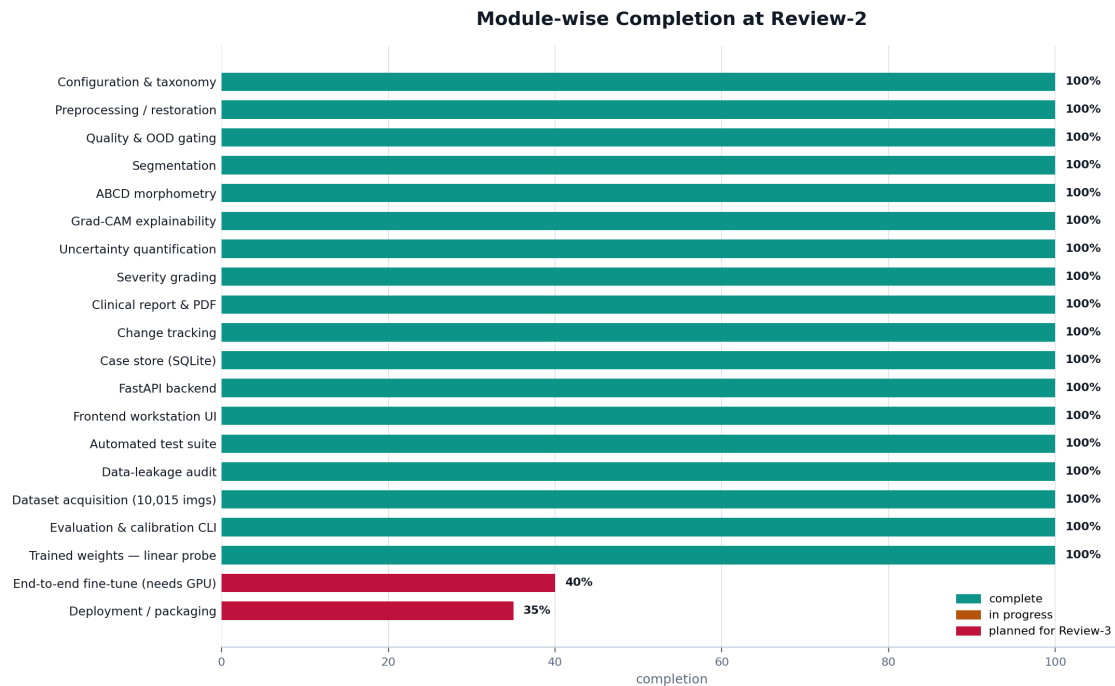


Figure 4 — Per-module completion at Review-2.

7.2 Honest limitations

- **The weights are a linear probe, not a fine-tuned model.** 53.7% balanced accuracy is well short of what this architecture can reach. Precision on the rare classes is poor — dermatofibroma 0.065, vascular 0.139 — meaning the model over-flags them. Fine-tuning the backbone on a GPU is the single highest-value remaining task.
- **Skin-tone representativeness.** HAM10000 is dominated by fair-skinned European and Australian populations. Performance on darker skin is uncharacterised and probably worse. This is the most serious limitation for any real use.
- **Only seven categories.** Squamous cell carcinoma, amelanotic melanoma, infections and inflammatory dermatoses are forced into the nearest of the seven and will be wrong.
- **The D of ABCD is approximated** with classical texture filters rather than expert annotation, so it is a weak signal. A and B are faithful to the geometric rule; C is a colour-quantisation approximation.
- **Segmentation is classical, not learned**, because the main HAM10000 release ships no masks. It reports its own confidence and falls back to an ellipse.
- **Absolute lesion size cannot be recovered** from a photograph without a scale reference, so change tracking reports size relative to the frame unless a field-of-view width is supplied.

7.3 Plan for Review-3

Area	Work	Acceptance criterion
Model	Fine-tune the full backbone end to end on Kaggle, Colab or a CUDA GPU	Balanced accuracy and rare-class precision both materially above the linear probe's 53.7% / 0.065
Verification	Re-run <code>verify_checkpoint.py --images</code>	21/21 checks pass; per-class recall confirms class order is not permuted

Area	Work	Acceptance criterion
Metrics	Regenerate docs/evaluation.json, the confusion matrix and calibration figures	Both remaining self-reported rows either replaced by measured or stated as unverified
Testing	Add trained-model integration tests and per-class recall floors	Suite fails if melanoma recall regresses below an agreed threshold
Performance	ONNX export or quantisation	Full-pipeline CPU latency below the current 2.9 s per image
Deployment	Containerise, enable DERM_API_KEY, serve over HTTPS, add structured logging	Reachable from another machine with authentication enforced
Documentation	Final project report and user guide	Reproducible from a clean checkout following section 4 alone

8. References

- 1 Tschandl P., Rosendahl C., Kittler H. *The HAM10000 dataset, a large collection of multi-source dermoscopic images of common pigmented skin lesions*. Scientific Data 5, 180161 (2018). doi:10.1038/sdata.2018.161
- 2 Harvard Dataverse, HAM10000 distribution. doi:10.7910/DVN/DBW86T (CC BY-NC 4.0).
- 3 Tan M., Le Q. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML 2019.
- 4 Selvaraju R. R. et al. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. ICCV 2017.
- 5 Chattopadhyay A. et al. *Grad-CAM++: Improved Visual Explanations for Deep Convolutional Networks*. WACV 2018.
- 6 Gal Y., Ghahramani Z. *Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning*. ICML 2016.
- 7 Houthby N. et al. *Bayesian Active Learning for Classification and Preference Learning*. arXiv:1112.5745 (2011).
- 8 Guo C. et al. *On Calibration of Modern Neural Networks*. ICML 2017.
- 9 Stolz W. et al. *ABCD rule of dermatoscopy: a new practical method for early recognition of malignant melanoma*. European Journal of Dermatology, 1994.
- 10 Telea A. *An Image Inpainting Technique Based on the Fast Marching Method*. Journal of Graphics Tools, 2004.
- 11 Finlayson G., Trezzi E. *Shades of Gray and Colour Constancy*. Color Imaging Conference, 2004.
- 12 Wightman R. *PyTorch Image Models (timm)*. github.com/huggingface/pytorch-image-models

This report was generated by scripts/make_project_report.py. Figures 1–4 are generated by scripts/make_diagrams.py. Both regenerate deterministically from the repository, so nothing in this document is a hand-maintained copy that can drift out of date.